Jira 2.0

*From **configurability** to **cognitive efficiency.***

PARTH AGGARWAL
Product Manager

THESIS

Jira's design philosophy is the problem.

Jira today is a system of record optimized for configurability and control. That depth is real and worth preserving - it is why the enterprise installed base hasn't moved.

Jira 2.0 introduces an AI interaction layer that makes it feel like a system of action - without compromising the underlying structure.

01 **Action-First Home**

Replace the activity feed with a surface that answers: what blocks others, what blocks me, what's overdue.

02 **Conversational Tickets**

Drafts a structured ticket from natural language. User confirms. Maker-checker preserved.

03 **Natural-Language Dashboards**

Ad-hoc exploration via NL. Structured JQL dashboards remain the source of truth.

STEP 1 · PERSONAS

Five users. One product. Five conflicting workflows.

PERSONA	PRIMARY USE CASES	TOP PAIN POINT
Software Engineer	Receive tickets, update status, link code, log work	*Status updates feel like overhead. Context-switching breaks flow.*
Product Manager	Backlog grooming, sprint planning, dependency tracking, status reporting	*Building any dashboard requires JQL fluency. Reporting goes to spreadsheets. Main issue - time spent.*
Engineering Manager	Sprint health, velocity tracking, blocker resolution, capacity planning	*No clean view of what's blocked across the team without configuration.*
QA Engineer	Bug reporting, test case tracking, regression management	*8–12 fields per bug. Batch creation is painful. Capture rate suffers. Causes delays.*
Executive	Portfolio health, milestone tracking, cross-team status	*No portfolio-level view without admin-level setup.*

STEP 1 · THE FRICTION, IN PRACTICE

Three real moments from a working week.

ENGINEER

The fourth context switch

Rahul gets a Jira mention. Navigates to ticket. Reads context. Switches to Slack to clarify. Switches to code. Updates status. Comments. Jira is the third or fourth context switch in his actual workflow - never the first.

PM

Ten seconds, fifteen minutes

I need to know how many tickets the team raised for Qure under the data charter in Q1. Today: build JQL filter, save it, build dashboard, configure gadget, render. Fifteen minutes. The question itself took ten seconds to ask.

MANAGER

What's blocked today?

Engineering manager opens the Jira home screen. Sees recommended spaces, spaces you might like, try Jira Service Management. None of it answers the question they came to answer: what's blocked on my team today?

STEP 2 · WHAT GETS USED, WHAT DOESN'T

Feature usage maps the gap between intent and reality.

FEATURE	USAGE	WHY
Issue creation	HIGH	Core workflow but high per-ticket friction
Sprint boards	HIGH	Daily standup tool, well-loved
Backlog management	HIGH	Core PM workflow
Dashboards	MEDIUM	Configuration overhead suppresses adoption
JQL filters	MEDIUM	Steep learning curve, power-users only
Automation rules	LOW	Discoverability and complexity barrier
Atlassian Intelligence	LOW	AI feels bolted-on, not workflow-native. Mostly used for summaries.
Mobile app	LOW	Limited functionality versus web

CAPABILITY GAPS

What's missing today.

- Conversational ticket creation
- Natural-language dashboards
- Context-aware home screen
- Native bug auto-triage (with insights into similar previous bugs)
- Cross-project dependency intelligence
- Meeting-to-ticket conversion

STEP 3 · THE THREE OPPORTUNITIES

Same data layer. Three new interaction surfaces.

01 Action-First Home

PROBLEM

Jira home is an activity feed and recommendation surface. Power users build personal filters as their actual home, bypassing the default entirely. Wasted real-estate that prioritises marketing over core usage.

MECHANISM

Surfaces work via inferred urgency - mentions, recent activity, due dates, dependency links - not just stale priority fields. Each surfaced item explains why it appears so users can trust, validate, or override. In low-signal environments, the system falls back to deterministic views (assigned work, due dates).

Inference quality depends on signal density. Works best in active projects & clean data, degrades in low-activity backlogs.

02 Conversational Tickets

PROBLEM

Ticket creation requires 8–12 fields per ticket. Engineers batch-create because per-ticket overhead is too high. Bug capture rate suffers as a result.

MECHANISM

User describes the issue in a few signal-dense sentences. System drafts a structured ticket - fields inferred, ambiguity surfaced as options. User confirms before save. Maker-Checker flow mandatory. User is final creator.

Assisted structuring, not autonomous creation. Removes input effort. Preserves user control. Saves time. Reduces cognitive load.

03 NL Dashboards

PROBLEM

Asking a question in Jira takes 10 seconds. Building a dashboard to answer it takes 15 minutes - JQL, filters, gadgets, configuration.

MECHANISM

User describes the question. System generates JQL and visualization, shows the interpretation, lets user refine or correct conversationally. Structured JQL-based dashboards remain source of truth for compliance and org-wide reporting.

Reduces query friction, not data hygiene. If underlying data is wrong, both NL and JQL fail. Frequently used queries can be saved and promoted into reusable dashboards

STEP 3 · SECONDARY OPPORTUNITIES (long-term roadmap)

The shared AI layer unlocks a wider product surface.

Auto-triage incoming bugs

AI suggests priority, assignee, and surfaces similar past issues. Human approves.

Smart sprint planning

Proposes sprint composition based on velocity history, dependencies, and team capacity.

Cross-project dependency graph

Visual dependency map auto-generated from ticket links. No manual graph maintenance.

Meeting-to-ticket conversion

Paste meeting notes; system extracts action items as draft tickets for review.

Mobile-first core workflows

Reimagine the engineer's daily Jira interactions natively for mobile, not as a desktop port.

AI failure handling

All AI features ship with one-click override. **Correction rate is a primary quality metric**, fed back to improve inference.

STEP 4 · VALIDATION

Each thesis ships with explicit success and failure signals.

HYPOTHESIS	PRIMARY METRIC	VALIDATES IF	FAILURE SIGNAL
Action-first home reduces navigation time	Time-to-first-action; target <60s vs current ~3min	Median TTFA drops 60%+ across all personas	*Users ignore AI-prioritized items → low trust in inferred signals*
Conversational ticket creation increases throughput	Tickets created/user takes lesser than 50% the time it took before	Creation time reduced +50% with no degradation in ticket quality	*High edit rate post AI draft → poor field inference*
NL dashboards expand analytics adoption	Weekly active dashboard creators (target: 2x current) & time taken to create a dashboard	Non-technical users create dashboards at meaningful rate	*High abandonment of NL dashboards → intent–output mismatch*

PRINCIPLES: Every feature ships behind a flag. Counter-metrics weighted equally to primary metrics. Ten enterprise customer interviews per quarter capture qualitative friction.

STEP 5 · ROADMAP & PRIORITIZATION

12-month plan. 10 engineers, 2 designers.

Q1 — FOUNDATION

- Action-first home MVP (FE + design heavy)
- Shared AI interpretation layer for ticket creation and query generation (BE + FS)
- Telemetry, beta program setup

Q2 — CONVERSATIONAL

- Conversational ticket creation V1 (FS + design)
- Natural-language dashboards V1 (FS + BE query engine)
- A/B infrastructure; 10% customer rollout

Q3 — ITERATION

- Iterate based on Q2 telemetry and customer feedback
- Auto-triage and smart prioritization (AI layer expansion)
- Mobile experience for action-first home

Q4 — SCALE & POLISH

- General availability for all customers
- Cross-project dependency intelligence
- Performance hardening; preserve power-user customization

STEP 6 · ROLE-BASED CUSTOMIZATION

Same data layer. Three native experiences.

ENGINEER

HOME DEFAULT

Assigned tickets sorted by sprint priority. WIP, blocked, & TBD items at top.

TICKET DETAIL

Code-centric ticket view: linked PRDs, PRs, branches, deployment status front-and-center.

HIDDEN BY DEFAULT

Roadmaps, advanced reports, portfolio views.

PRODUCT MANAGER

HOME DEFAULT

Items blocked on me. Dependencies on me. Overdue items. Sprint health summary.

TICKET DETAIL

Business-context-centric: customer, value, dependencies up top. Code linkage collapsible.

HIDDEN BY DEFAULT

Detailed code commit threads.

ENGINEERING MANAGER

HOME DEFAULT

Team velocity. Blockers across team. Capacity utilization. Open escalations.

TICKET DETAIL

Owner-and-status-centric. Aggregated team-level views by default.

HIDDEN BY DEFAULT

Individual ticket commentary noise.

These views are defaults, not constraints. Users can override, customize, and switch views at any time.

STEP 7 · PROTOTYPE

Working prototype. Deployed. Clickable.

Built & deployed on Github as a working product experience & not static mocks to demonstrate real interaction flows.

Repo and code are open-source - publicly inspectable and reusable (https://github.com/Parth8/jira2).

Note - The prototype was built using AI-assisted development (Claude) instead of purely no-code tools like Lovable. This decision was intentional - to ensure deeper interaction fidelity (state, flows, failure handling, persona-based rendering, etc) that low-code tools struggled to support reliably.

(The goal was not the tool, but to demonstrate a working system that reflects the vision)

LIVE URL

`https://parth8.github.io/jira2/`

01 **Action-First Home**

Persona toggle. Same data, three default views.

02 **Conversational Ticket**

NL input ⟶ drafted ticket ⟶ user confirms.

03 **NL Dashboard**

Type the question. See generated query. Render.

04 **Role-Based Views**

Engineer / PM / Manager - switchable.

STEPS 8 & 9 · GTM & RELAUNCH

Land in the existing installed base. Win on friction.

The hook? Immediate time savings on daily workflows - users experience value on day one without needing migration or retraining.

TARGET & ROLLOUT

Target

Existing Atlassian Cloud customers. Mid-market and enterprise (500–10,000 user accounts) - the segment feeling Jira's complexity most acutely.

Phase 1 · Months 1-2

Beta with 50 design partners. Free participation. Weekly feedback cycles.

Phase 2 · Months 3-4

Limited GA at 10% of cloud customers. Opt-in. Telemetry collection. Refinement loop.

Phase 3 · Months 5-6

Full GA. Migration tooling. Same-price upgrade for existing tiers; AI features bundled in Premium and Enterprise.

POSITIONING

Jira, simplified. AI-native. Built for how teams actually work in 2026.

vs Linear Linear's simplicity, with enterprise depth and your existing data.

vs Monday/ClickUp Built for engineering-first teams, not generic project management.

vs Status-quo Jira Same data, same integrations, dramatically less friction.

No migration required - sits on top of existing data, workflows, and integrations.

STEP 10 · ANALYTICS & SUCCESS PLAN

Four metric quadrants. One executive dashboard.

ADOPTION

- Jira 2.0 home as % of total Jira DAU
- Conversational tickets as % of all tickets created
- NL dashboards as % of all dashboards created

ENGAGEMENT

- Time-to-first-action on home (median, p90)
- Tickets per user per week
- Persona view distribution (Engineer / PM / Manager)

OUTCOMES

- Ticket cycle time (creation → resolution)
- Reporting time reduction (customer survey)
- NPS shift on Jira product

HEALTH (counter-metrics)

- AI parsing accuracy: post-creation edit rate <15%
- NL dashboard query accuracy ≥90%
- Feature flag rollback rate near zero

Single executive dashboard: four quadrants. Updated weekly during beta, daily post-GA. Drill-down per persona and per customer segment.

CLOSE

Why this matters.

Jira has 100K+ enterprise customers and is the default project management tool for engineering organizations globally.

It is also one of the most complained-about products in the same audience.

The opportunity is not to compete with Linear on simplicity for new customers. The opportunity is to dramatically reduce friction for the existing installed base by leveraging AI to handle the configuration overhead Jira has accumulated over twenty years.

> ***Jira 2.0 is not a rebuild. It is an AI interaction layer on top of Jira's existing system of record - preserving structure while dramatically reducing the effort required to use it.***